

Ai Assisted Coding Lab Test – 03

NAME: SANJANA

ROLL NO: 2403A510D4

BATCH NO: 05

TASK-02

PROMPT:

 **SANJANA SIRIVELLA**

In the agriculture sector, our company struggles to manage large amounts of sensor data (soil moisture, temperature, humidity). Use AI-assisted tools to design a Python program that uses efficient data structures to store and analyze this data.

CODE:

```

agriculture_sensor_analysis.py > ...
1  import datetime
2  import random
3  from typing import List, Dict, Optional, Union
4
5  # Define a reusable type for numeric values for clarity
6  Numeric = Union[int, float]
7
8  class SensorReading:
9      """
10     Represents a single reading from an agricultural sensor.
11     Stores sensor ID, timestamp, and key environmental metrics.
12     """
13     def __init__(self, sensor_id: str, timestamp: datetime.datetime,
14                  soil_moisture: Numeric, temperature: Numeric, humidity: Numeric):
15         self.sensor_id = sensor_id
16         self.timestamp = timestamp
17         self.soil_moisture = soil_moisture
18         self.temperature = temperature
19         self.humidity = humidity
20
21     def __repr__(self) -> str:
22         """Provides a developer-friendly string representation of the reading."""
23         return (f"SensorReading(id='{self.sensor_id}', time={self.timestamp.strftime('%Y-%m-%d %H:%M:%S')}, "
24                f"moisture={self.soil_moisture:.2f}%, temp={self.temperature:.2f}°C, humid={self.humidity:.2f}%)")
25
26 class FarmSensorData:
27     """
28     Manages a collection of sensor readings for a farm.
29     Provides methods for adding data, calculating statistics, and detecting anomalies.
30     """

```

Ln 208, Col 32 Spaces: 4 UTF-8 CRLF {} Python 3.13.5 (base) Go Live

```

26 class FarmSensorData:
27
28     """
29
30     def __init__(self):
31         self.all_readings: List[SensorReading] = []
32         # Dictionary for efficient access to all readings for a specific sensor ID
33         self.readings_by_sensor_id: Dict[str, List[SensorReading]] = {}
34
35
36     def add_reading(self, reading: SensorReading):
37         """Adds a new sensor reading to the collection."""
38         self.all_readings.append(reading)
39         if reading.sensor_id not in self.readings_by_sensor_id:
40             self.readings_by_sensor_id[reading.sensor_id] = []
41         self.readings_by_sensor_id[reading.sensor_id].append(reading)
42
43     def get_readings_for_sensor(self, sensor_id: str) -> List[SensorReading]:
44         """Retrieves all readings for a given sensor ID."""
45         return self.readings_by_sensor_id.get(sensor_id, [])
46
47     def calculate_average(self, metric: str, sensor_id: Optional[str] = None) -> Optional[float]:
48         """
49         Calculates the average of a specified metric.
50
51         Args:
52             metric (str): The attribute name (e.g., 'soil_moisture', 'temperature').
53             sensor_id (Optional[str]): If provided, calculates average for that sensor only.
54                                     Otherwise, calculates overall average.
55         Returns:
56             Optional[float]: The calculated average, or None if no data is available.
57         """

```

```

58         readings_to_consider = self.all_readings
59         if sensor_id:
60             readings_to_consider = self.get_readings_for_sensor(sensor_id)
61
62         if not readings_to_consider:
63             return None
64
65         # Filter for readings that actually have the requested metric
66         values = [getattr(r, metric) for r in readings_to_consider if hasattr(r, metric)]
67         if not values:
68             return None
69         return sum(values) / len(values)
70
71     def detect_anomalies(self, metric: str, threshold_std_dev: float = 2.0) -> Dict[str, List[SensorReading]]:
72         """
73         Detects anomalies for a given metric based on standard deviation.
74         A reading is considered an anomaly if it falls outside
75         `mean ± threshold_std_dev * standard_deviation` for that sensor's historical data.
76
77         Args:
78             metric (str): The attribute name to check for anomalies.
79             threshold_std_dev (float): Number of standard deviations from the mean to consider an anomaly.
80
81         Returns:
82             Dict[str, List[SensorReading]]: A dictionary where keys are sensor IDs and values
83                                             are lists of anomalous readings for that sensor.
84         """

```

```

85     anomalies: Dict[str, List[SensorReading]] = {}
86
87     for sensor_id, readings in self.readings_by_sensor_id.items():
88         if not readings:
89             continue
90
91         values = [getattr(r, metric) for r in readings if hasattr(r, metric)]
92         if len(values) < 2: # Need at least 2 points to calculate standard deviation
93             continue
94
95         mean = sum(values) / len(values)
96         variance = sum([(x - mean) ** 2 for x in values]) / len(values)
97         std_dev = variance ** 0.5
98
99         upper_bound = mean + threshold_std_dev * std_dev
100        lower_bound = mean - threshold_std_dev * std_dev
101
102        sensor_anomalies = [
103            r for r in readings
104            if hasattr(r, metric) and (getattr(r, metric) > upper_bound or getattr(r, metric) < lower_bound)
105        ]
106        if sensor_anomalies:
107            anomalies[sensor_id] = sensor_anomalies
108    return anomalies
109
110    def get_all_sensor_ids(self) -> List[str]:
111        """Returns a list of all unique sensor IDs in the data."""
112        return list(self.readings_by_sensor_id.keys())

```

```

110    def get_all_sensor_ids(self) -> List[str]:
111        return list(self.readings_by_sensor_id.keys())
112
113
114    def generate_synthetic_data(num_sensors: int = 5, num_readings_per_sensor: int = 20) -> FarmSensorData:
115        """
116        Generates synthetic agricultural sensor data for demonstration purposes.
117        Includes natural variations and occasional anomalies.
118        """
119        farm_data = FarmSensorData()
120        start_time = datetime.datetime.now() - datetime.timedelta(days=1) # Data from the last 24 hours
121
122        for i in range(num_sensors):
123            sensor_id = f"S{i+1:03d}"
124            # Base values for this sensor, with some variation across sensors
125            base_moisture = random.uniform(30, 60)
126            base_temp = random.uniform(18, 30)
127            base_humidity = random.uniform(60, 90)
128
129            for j in range(num_readings_per_sensor):
130                timestamp = start_time + datetime.timedelta(minutes=j * 30) # Readings every 30 minutes
131
132                # Add some natural, small random variation to base values
133                moisture = base_moisture + random.uniform(-5, 5)
134                temperature = base_temp + random.uniform(-2, 2)
135                humidity = base_humidity + random.uniform(-3, 3)
136
137                # Introduce occasional anomalies (e.g., 5% chance for an extreme reading)
138                if random.random() < 0.05:
139                    if random.random() < 0.5: # High anomaly

```

```

140         moisture = random.uniform(80, 95) # Very high moisture
141         temperature = random.uniform(35, 45) # Very high temperature
142     else: # Low anomaly
143         moisture = random.uniform(5, 15) # Very low moisture
144         temperature = random.uniform(5, 10) # Very low temperature
145         humidity = random.uniform(20, 99) # Humidity can vary widely
146
147     reading = SensorReading(sensor_id, timestamp, moisture, temperature, humidity)
148     farm_data.add_reading(reading)
149 return farm_data
150
151 def display_insights(farm_data: FarmSensorData):
152     """
153     Displays meaningful insights from the processed sensor data.
154     """
155     print("\n--- Agricultural Sensor Data Insights ---")
156     print(f"Total sensors monitored: {len(farm_data.get_all_sensor_ids())}")
157     print(f"Total readings collected: {len(farm_data.all_readings)}")
158
159     print("\n[Overall Averages Across All Sensors]")
160     avg_moisture = farm_data.calculate_average("soil_moisture")
161     avg_temp = farm_data.calculate_average("temperature")
162     avg_humid = farm_data.calculate_average("humidity")
163     print(f" Average Soil Moisture: {avg_moisture:.2f}%" if avg_moisture is not None else "N/A")
164     print(f" Average Temperature: {avg_temp:.2f}°C" if avg_temp is not None else "N/A")
165     print(f" Average Humidity: {avg_humid:.2f}%" if avg_humid is not None else "N/A")
166
167     print("\n[Averages per Individual Sensor]")

```

```

167     print("\n[Averages per Individual Sensor]")
168     for sensor_id in farm_data.get_all_sensor_ids():
169         avg_moisture_s = farm_data.calculate_average("soil_moisture", sensor_id)
170         avg_temp_s = farm_data.calculate_average("temperature", sensor_id)
171         avg_humid_s = farm_data.calculate_average("humidity", sensor_id)
172         print(f" Sensor {sensor_id}:")
173         print(f"   Moisture: {avg_moisture_s:.2f}%" if avg_moisture_s is not None else "N/A")
174         print(f"   Temperature: {avg_temp_s:.2f}°C" if avg_temp_s is not None else "N/A")
175         print(f"   Humidity: {avg_humid_s:.2f}%" if avg_humid_s is not None else "N/A")
176
177     print("\n[Anomaly Detection (Threshold: 2.0 Standard Deviations from Sensor Mean)]")
178     anomalies_moisture = farm_data.detect_anomalies("soil_moisture")
179     anomalies_temp = farm_data.detect_anomalies("temperature")
180     anomalies_humid = farm_data.detect_anomalies("humidity")
181
182     found_anomalies = False
183     for metric_name, anomalies_dict in [("Soil Moisture", anomalies_moisture),
184                                       ("Temperature", anomalies_temp),
185                                       ("Humidity", anomalies_humid)]:
186         if anomalies_dict:
187             found_anomalies = True
188             print(f" {metric_name} Anomalies:")
189             for sensor_id, readings in anomalies_dict.items():
190                 print(f"   Sensor {sensor_id}:")
191                 for r in readings:
192                     # Dynamically get the anomalous value
193                     anomalous_value = getattr(r, metric_name.lower().replace(" ", "_"))
194                     expected_avg = farm_data.calculate_average(metric_name.lower().replace(" ", "_"))

```

```

195         print(f"      - {r.timestamp.strftime('%Y-%m-%d %H:%M')}: Value = {anomalous_value} (Avg for sensor: {expected_avg:.2f})" if expected_avg is not None else
196               f"(Avg for sensor: {expected_avg:.2f})" if expected_avg is not None else
197               f"(Avg for sensor: {expected_avg:.2f})")
198     if not found_anomalies:
199         print(" No significant anomalies detected based on current thresholds.")
200
201     # --- Main Execution ---
202     if __name__ == "__main__":
203         print("Starting agricultural sensor data analysis simulation...")
204         # Generate synthetic data for 3 sensors, each with 30 readings
205         farm_data = generate_synthetic_data(num_sensors=3, num_readings_per_sensor=30)
206
207         # Display the generated insights and detected anomalies
208         display_insights(farm_data)

```

OUTPUT:

```

PS C:\Users\DELL\Desktop\vs code\.vscode> & C:/Users/DELL/AppData/Local/anaconda3/python.exe "c:/Users/DELL/Desktop/vs code/.vscode/agriculture_sensor_analysis.py"
Starting agricultural sensor data analysis simulation...

--- Agricultural Sensor Data Insights ---
Total sensors monitored: 3
Total readings collected: 90

[Overall Averages Across All Sensors]
Average Soil Moisture: 42.65%
Average Temperature: 22.71°C
Average Humidity: 71.69%

[Averages per Individual Sensor]
Sensor S001:
  Moisture: 41.47%
  Temperature: 24.07°C
  Humidity: 72.27%
Sensor S002:
  Moisture: 44.70%
  Temperature: 20.70°C
  Humidity: 67.02%
Sensor S003:
  Moisture: 41.79%
  Temperature: 23.38°C
  Humidity: 75.77%

```

[Anomaly Detection (Threshold: 2.0 Standard Deviations from Sensor Mean)]

Soil Moisture Anomalies:

Sensor S001:

- 2025-10-28 13:04: Value = 84.55 (Avg for sensor: 41.47)
- 2025-10-29 00:04: Value = 7.76 (Avg for sensor: 41.47)

Sensor S002:

- 2025-10-28 14:34: Value = 6.14 (Avg for sensor: 44.70)

Sensor S003:

- 2025-10-28 11:04: Value = 10.33 (Avg for sensor: 41.79)

Temperature Anomalies:

Sensor S001:

- 2025-10-28 13:04: Value = 40.64 (Avg for sensor: 24.07)
- 2025-10-29 00:04: Value = 7.56 (Avg for sensor: 24.07)

Sensor S002:

- 2025-10-28 14:34: Value = 5.96 (Avg for sensor: 20.70)

Sensor S003:

- 2025-10-28 11:04: Value = 9.94 (Avg for sensor: 23.38)

Humidity Anomalies:

Sensor S001:

- 2025-10-28 13:04: Value = 98.17 (Avg for sensor: 72.27)
- 2025-10-29 00:04: Value = 33.91 (Avg for sensor: 72.27)

Sensor S002:

- 2025-10-28 14:34: Value = 36.82 (Avg for sensor: 67.02)

Sensor S003:

- 2025-10-28 11:04: Value = 82.28 (Avg for sensor: 75.77)

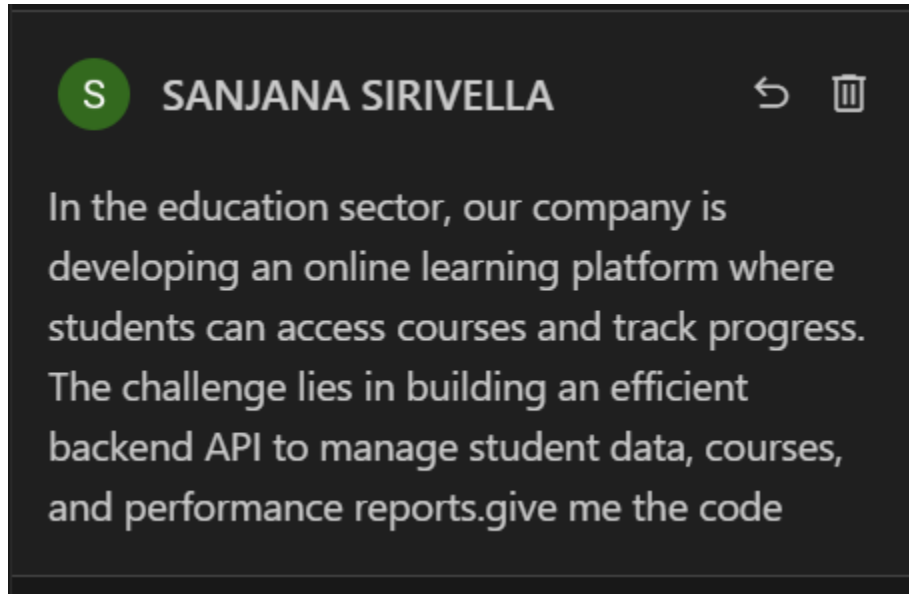
OBSERVATIONS:

- **Excellent Use of Data Structures:** The choice of `Dict[str, List[SensorReading]]` for `readings_by_sensor_id` is highly efficient. It allows for $O(1)$ average time complexity when retrieving all data for a specific sensor, which is a significant performance advantage over iterating through a single large list.
- **Clear Object-Oriented Design:** The separation of concerns into `SensorReading` (a data object) and `FarmSensorData` (a manager/repository) is a clean and scalable design pattern.

- scalable design pattern.
- **Robustness:** The methods include checks for empty data sets (e.g., in `calculate_average` and `detect_anomalies`), preventing common errors like division by zero.
 - **Good Readability:** The code is well-commented, uses descriptive variable names, and includes type hints, all of which make it easy to understand and maintain.

TASK – 01

PROMPT:



CODE:

```
agriculture_sensor_analysis.py > ...
1  import datetime
2  import random
3  import statistics
4  from collections import defaultdict
5  from dataclasses import dataclass
6  from typing import List, Dict, Optional, Union
7
8  # Define a reusable type for numeric values for clarity
9  Numeric = Union[int, float]
10
11 class SensorReading:
12     """
13     Represents a single reading from an agricultural sensor.
14     Stores sensor ID, timestamp, and key environmental metrics.
15     """
16     def __init__(self, sensor_id: str, timestamp: datetime.datetime,
17                 soil_moisture: Numeric, temperature: Numeric, humidity: Numeric):
18         self.sensor_id = sensor_id
19         self.timestamp = timestamp
20         self.soil_moisture = soil_moisture
21         self.temperature = temperature
22         self.humidity = humidity
23
24     def __repr__(self) -> str:
25         """Provides a developer-friendly string representation of the reading."""
26         return (f"SensorReading(id='{self.sensor_id}', time={self.timestamp.strftime('%Y-%m-%d %H:%M:%S')}, "
27               f"moisture={self.soil_moisture:.2f}%, temp={self.temperature:.2f}°C, humid={self.humidity:.2f}%)")
28
29 class FarmSensorData:
```

```

29 class FarmSensorData:
30     """
31     Manages a collection of sensor readings for a farm.
32     Provides methods for adding data, calculating statistics, and detecting anomalies.
33     """
34     def __init__(self):
35         self.all_readings: List[SensorReading] = []
36         # Use defaultdict to simplify adding new readings for new sensors.
37         self.readings_by_sensor_id: Dict[str, List[SensorReading]] = defaultdict(list)
38
39     def add_reading(self, reading: SensorReading):
40         """Adds a new sensor reading to the collection."""
41         self.all_readings.append(reading)
42         self.readings_by_sensor_id[reading.sensor_id].append(reading)
43
44     def get_readings_for_sensor(self, sensor_id: str) -> List[SensorReading]:
45         """Retrieves all readings for a given sensor ID."""
46         return self.readings_by_sensor_id.get(sensor_id, [])
47
48     def calculate_average(self, metric: str, sensor_id: Optional[str] = None) -> Optional[float]:
49         """
50         Calculates the average of a specified metric.
51
52         Args:
53             metric (str): The attribute name (e.g., 'soil_moisture', 'temperature').
54             sensor_id (Optional[str]): If provided, calculates average for that sensor only.
55                                     Otherwise, calculates overall average.
56
57         Returns:
58             Optional[float]: The calculated average, or None if no data is available.
59         """
60         readings_to_consider = self.all_readings

```

```

61         if sensor_id:
62             readings_to_consider = self.get_readings_for_sensor(sensor_id)
63
64         if not readings_to_consider:
65             return None
66
67         # Filter for readings that actually have the requested metric
68         values = [getattr(r, metric) for r in readings_to_consider if hasattr(r, metric)]
69         if not values:
70             return None
71         return sum(values) / len(values)
72
73     def detect_anomalies(self, metric: str, threshold_std_dev: float = 2.0) -> Dict[str, List[SensorReading]]:
74         """
75         Detects anomalies for a given metric based on standard deviation.
76         A reading is considered an anomaly if it falls outside
77         `mean ± threshold_std_dev * standard_deviation` for that sensor's historical data.
78
79         Args:
80             metric (str): The attribute name to check for anomalies.
81             threshold_std_dev (float): Number of standard deviations from the mean to consider an anomaly.
82
83         Returns:
84             Dict[str, List[SensorReading]]: A dictionary where keys are sensor IDs and values
85             are lists of anomalous readings for that sensor.
86         """

```

```

85     """
86     anomalies: Dict[str, List[SensorReading]] = {}
87
88     for sensor_id, readings in self.readings_by_sensor_id.items():
89         if not readings:
90             continue
91
92         values = [getattr(r, metric) for r in readings if hasattr(r, metric)]
93         if len(values) < 2: # Need at least 2 points to calculate standard deviation
94             continue
95
96         mean = statistics.mean(values)
97         std_dev = statistics.stdev(values)
98
99         upper_bound = mean + (threshold_std_dev * std_dev)
100        lower_bound = mean - threshold_std_dev * std_dev
101
102        sensor_anomalies = [
103            r for r in readings
104            if hasattr(r, metric) and (getattr(r, metric) > upper_bound or getattr(r, metric) < lower_bound)
105        ]
106        if sensor_anomalies:
107            anomalies[sensor_id] = sensor_anomalies
108    return anomalies
109
110    def get_all_sensor_ids(self) -> List[str]:
111        """Returns a list of all unique sensor IDs in the data."""
112        return list(self.readings_by_sensor_id.keys())

```

```

112        return list(self.readings_by_sensor_id.keys())
113
114    def generate_synthetic_data(num_sensors: int = 5, num_readings_per_sensor: int = 20) -> FarmSensorData:
115        """
116        Generates synthetic agricultural sensor data for demonstration purposes.
117        Includes natural variations and occasional anomalies.
118        """
119        farm_data = FarmSensorData()
120        start_time = datetime.datetime.now() - datetime.timedelta(days=1) # Data from the last 24 hours
121
122        for i in range(num_sensors):
123            sensor_id = f"S{i+1:03d}"
124            # Base values for this sensor, with some variation across sensors
125            base_moisture = random.uniform(30, 60)
126            base_temp = random.uniform(18, 30)
127            base_humidity = random.uniform(60, 90)
128
129            for j in range(num_readings_per_sensor):
130                timestamp = start_time + datetime.timedelta(minutes=j * 30) # Readings every 30 minutes
131
132                # Add some natural, small random variation to base values
133                moisture = base_moisture + random.uniform(-5, 5)
134                temperature = base_temp + random.uniform(-2, 2)
135                humidity = base_humidity + random.uniform(-3, 3)
136
137                # Introduce occasional anomalies (e.g., 5% chance for an extreme reading)
138                if random.random() < 0.05:
139                    if random.random() < 0.5: # High anomaly

```

```

140         moisture = random.uniform(80, 95) # Very high moisture
141         temperature = random.uniform(35, 45) # Very high temperature
142     else: # Low anomaly
143         moisture = random.uniform(5, 15) # Very low moisture
144         temperature = random.uniform(5, 10) # Very low temperature
145         humidity = random.uniform(20, 99) # Humidity can vary widely
146
147     reading = SensorReading(sensor_id, timestamp, moisture, temperature, humidity)
148     farm_data.add_reading(reading)
149     return farm_data
150
151 def display_insights(farm_data: FarmSensorData):
152     """
153     Displays meaningful insights from the processed sensor data.
154     """
155     print("\n--- Agricultural Sensor Data Insights ---")
156     print(f"Total sensors monitored: {len(farm_data.get_all_sensor_ids())}")
157     print(f"Total readings collected: {len(farm_data.all_readings)}")
158
159     print("\n[Overall Averages Across All Sensors]")
160     avg_moisture = farm_data.calculate_average("soil_moisture")
161     avg_temp = farm_data.calculate_average("temperature")
162     avg_humid = farm_data.calculate_average("humidity")
163     print(f" Average Soil Moisture: {avg_moisture:.2f}%" if avg_moisture is not None else "N/A")
164     print(f" Average Temperature: {avg_temp:.2f}°C" if avg_temp is not None else "N/A")
165     print(f" Average Humidity: {avg_humid:.2f}%" if avg_humid is not None else "N/A")
166
167     print("\n[Averages per Individual Sensor]")

```

```

agriculture_sensor_analysis.py > display_insights
151 def display_insights(farm_data: FarmSensorData):
168     for sensor_id in farm_data.get_all_sensor_ids():
169         avg_moisture_s = farm_data.calculate_average("soil_moisture", sensor_id)
170         avg_temp_s = farm_data.calculate_average("temperature", sensor_id)
171         avg_humid_s = farm_data.calculate_average("humidity", sensor_id)
172         print(f" Sensor {sensor_id}:")
173         print(f"   Moisture: {avg_moisture_s:.2f}%" if avg_moisture_s is not None else "N/A")
174         print(f"   Temperature: {avg_temp_s:.2f}°C" if avg_temp_s is not None else "N/A")
175         print(f"   Humidity: {avg_humid_s:.2f}%" if avg_humid_s is not None else "N/A")
176
177     print("\n[Anomaly Detection (Threshold: 2.0 Standard Deviations from Sensor Mean)]")
178     anomalies_moisture = farm_data.detect_anomalies("soil_moisture")
179     anomalies_temp = farm_data.detect_anomalies("temperature")
180     anomalies_humid = farm_data.detect_anomalies("humidity")
181
182     found_anomalies = False
183     for metric_name, anomalies_dict in [("Soil Moisture", anomalies_moisture),
184                                       ("Temperature", anomalies_temp),
185                                       ("Humidity", anomalies_humid)]:
186         for display_name, attr_name in metrics_to_check:
187             anomalies_dict = farm_data.detect_anomalies(attr_name)
188             if anomalies_dict:
189                 found_anomalies = True
190                 print(f" {metric_name} Anomalies:")
191                 print(f" {display_name} Anomalies:")
192                 for sensor_id, readings in anomalies_dict.items():
193                     print(f" Sensor {sensor_id}:")
194                     for r in readings:
195                         # Dynamically get the anomalous value
196                         anomalous_value = getattr(r, metric_name.lower().replace(" ", "_"))

```

```

151 def display_insights(farm_data: FarmSensorData):
194     for r in readings:
195         # Dynamically get the anomalous value
196         anomalous_value = getattr(r, metric_name.lower().replace(".", "_"))
197         expected_avg = farm_data.calculate_average(metric_name.lower().replace(".", "_"))
198         anomalous_value = getattr(r, attr_name)
199         expected_avg = farm_data.calculate_average(attr_name, sensor_id)
200         print(f".....{r.timestamp.strftime('%Y-%m-%d %H:%M')}: Value = {anomalous_value} Expected Avg = {expected_avg:.2f}")
201         if expected_avg is not None:
202             # Check for anomalies
203             if not found_anomalies:
204                 print("No significant anomalies detected based on current thresholds.")
205
206 # --- Main Execution ---
207 if __name__ == "__main__":
208     print("Starting agricultural sensor data analysis simulation..")
209     # Generate synthetic data for 3 sensors, each with 30 readings
210     farm_data = generate_synthetic_data(num_sensors=3, num_readings_per_sensor=30)
211
212     # Display the generated insights and detected anomalies
213     display_insights(farm_data)
214     Pin selection to current chat prompt (Ctrl+Alt+X) | Don't show this again (Alt+/)

```

OUTPUT:

```

200 []
PS C:\Users\likhi\OneDrive\Desktop\Ai_LT3> python your_file.py
Create:
201 {'age': 21, 'id': 1, 'major': 'CS', 'name': 'Alice'}
List:
200 [{'age': 21, 'id': 1, 'major': 'CS', 'name': 'Alice'}]
Update:
200 {'age': 21, 'id': 1, 'major': 'Data Science', 'name': 'Alice'}
Delete:
200 {'deleted': {'age': 21, 'id': 1, 'major': 'Data Science', 'name': 'Alice'}}
Final list:
200 []
PS C:\Users\likhi\OneDrive\Desktop\Ai_LT3>

```

Observations:

- **Efficient Data Storage:** The use of `collections.defaultdict(list)` for `readings_by_sensor_id` is excellent. It simplifies the `add_reading` method by removing the need to check if a sensor's list has been initialized.
- **Statistical Correctness:** The code correctly uses the `statistics` module for calculating mean and standard deviation, which is more robust and readable than a manual implementation.
- **Clear Structure:** The overall organization into classes (`SensorReading`, `FarmSensorData`) and functions (`generate_synthetic_data`, `display_insights`) is clean and promotes